Trabajo Final de Grado - Gestion de Empresas Colaboradoras

- Autor: Luis Angel
- Tutora: Elena
- Fecha de revision: 17/03/2026
- Repositorio: https://github.com/lmendez861/TFG-Agora

1. Portada

Titulo: Gestion de Empresas Colaboradoras para FP Dual Autor: Luis Angel Tutora: Elena Fecha: 17/03/2026

2. Agradecimientos

Quiero agradecer a mi tutora Elena su seguimiento, sus observaciones y la orientacion prestada durante el desarrollo de este trabajo. Tambien agradezco al profesorado del ciclo y al centro educativo el contexto real aportado para enfocar el proyecto hacia una necesidad concreta y utilizable. Por ultimo, agradezco a mi entorno personal el apoyo y la constancia durante el proceso de analisis, implementacion, pruebas y redaccion final de la memoria.

3. Indice

En la maquetacion final se incluira indice general, indice de tablas e indice de figuras.

3.1 Indice de imagenes

1. Figura 1. Esquema de bloques de funcionalidad del sistema.
2. Figura 2. Esquema relacional de la base de datos.
3. Figura 3. Panel interno, vista dashboard con KPI y exportacion.
4. Figura 4. Panel interno, vista de solicitudes de empresa.
5. Figura 5. Portal externo para registro y seguimiento de solicitudes.

4. Resumen

4.1 Resumen (ES)

Este proyecto desarrolla una plataforma web para gestionar empresas colaboradoras, convenios, estudiantes, tutores y solicitudes externas en un entorno de FP Dual. La solucion combina una API en Symfony, un panel interno en React/TypeScript y un portal externo para empresas. El sistema permite CRUD de entidades principales, seguimiento documental, workflow de convenios, mensajeria asociada a solicitudes, exportacion CSV y supervision operativa del despliegue. La validacion tecnica actual confirma 61 pruebas backend superadas, 13 pruebas frontend y compilacion correcta de la build integrada.

Palabras clave: FP Dual, empresas colaboradoras, convenios, Symfony, React, gestion academica.

4.2 Summary (EN)

This project delivers a web platform to manage partner companies, agreements, students, mentors, and external registration requests for dual training. The solution combines a Symfony API, an internal

React/TypeScript dashboard, and an external company portal. The system supports CRUD operations for the main entities, document handling, agreement workflow, request messaging, CSV exports, and operational supervision for demo deployment. Current technical validation confirms 61 passing backend tests, 13 frontend tests, and successful integrated production builds.

Keywords: dual training, partner companies, agreements, Symfony, React, academic management.

5. Antecedentes / Introduccion

- La gestion inicial de empresas colaboradoras, convenios y practicas se hacia con hojas de calculo, correos y documentos dispersos.
- Ese enfoque generaba duplicidades, falta de trazabilidad, poca visibilidad de estados y dependencia de conocimiento manual.
- El proyecto Agora original se reutilizo solo como base tecnica. La logica previa de chats y bots se archivo en legacy/.
- El nuevo alcance se centra en un problema real del centro: registrar empresas, aprobar solicitudes, formalizar convenios y controlar asignaciones y seguimientos.
- El sistema esta orientado a coordinacion de practicas, tutores academicos, tutores profesionales y empresas interesadas en colaborar.

6. Objetivos y alcance

6.1 Objetivo general

Disenar e implantar una aplicacion web que centralice la gestion de empresas colaboradoras y practicas formativas, unificando informacion operativa, control documental y seguimiento de estados en una unica plataforma.

6.2 Objetivos especificos

1. Digitalizar el ciclo de vida de empresa, convenio, estudiante, tutor y asignacion.
2. Habilitar un flujo publico de solicitud de empresa con verificacion y aprobacion interna.
3. Ofrecer un panel interno con KPI, tablas de gestion, detalle de entidades y acciones CRUD.
4. Incorporar workflow documental y alertas para convenios y empresas.
5. Proteger la API y el panel con autenticacion y jerarquia de roles.

6.3 Alcance

- Dentro del alcance: panel interno, portal externo, API REST, base de datos relacional, usuarios demo, solicitudes de empresas, mensajeria vinculada, documentos, checklist, alertas y exportacion CSV de listados operativos.
- Fuera del alcance actual: firma electronica avanzada, integracion con ERP o SGA, exportacion PDF maquetada desde la aplicacion, almacenamiento documental en nube y pruebas E2E completas.

7. Definiciones

- Empresa colaboradora: organizacion que puede acoger estudiantes del centro.

- Convenio: acuerdo que formaliza la colaboracion entre el centro y una empresa.
- Asignacion: relacion operativa entre estudiante, convenio, empresa y tutores.
- Seguimiento: registro de reuniones, incidencias o acciones de control sobre una asignacion.
- Solicitud de empresa: alta inicial enviada por una empresa desde el portal externo antes de ser aprobada.

8. Notaciones y simbolos

- API: Application Programming Interface.
- CRUD: Create, Read, Update, Delete.
- FP Dual: Formacion Profesional Dual.
- KPI: Key Performance Indicator.
- RGPD: Reglamento General de Proteccion de Datos.
- SPA: Single Page Application.

9. Desarrollo del trabajo final

9.1 Analisis de requisitos

- Actores principales: coordinador o administrador, tutor academico, tutor profesional, estudiante y empresa externa.
- Casos de uso principales: consultar KPI, registrar empresas, crear y actualizar convenios, gestionar estudiantes y asignaciones, revisar solicitudes externas, adjuntar documentos, intercambiar mensajes y consultar detalle historico.
- Requisitos funcionales: listados con filtros, operaciones CRUD, validaciones de negocio, workflow de convenios, aprobacion y rechazo de solicitudes, detalle de entidades, notificaciones y consulta de tutores.
- Requisitos no funcionales: interfaz responsive, errores JSON normalizados, seguridad por roles, arquitectura mantenible, base de datos portable y despliegue reproducible en entorno local.

9.1.1 Planificacion y metodologia de trabajo

- Fase 1. Analisis del problema y definicion del alcance real para centrar el proyecto en empresas, convenios, estudiantes, asignaciones y solicitudes externas.
- Fase 2. Diseno tecnico de arquitectura, entidades y relaciones, separando API, panel interno y portal externo para reducir acoplamiento.
- Fase 3. Implementacion iterativa por modulos, validando primero CRUD nucleares y despues funcionalidades de valor como workflow de convenios, mensajeria y exportacion CSV.
- Fase 4. Pruebas tecnicas, correccion de incidencias y preparacion de documentacion de entrega, incluyendo memoria, anexos y guia de demo.

9.2 Diseno de la solucion

- La solucion se divide en tres piezas: backend Symfony 7.3, panel interno React 18 + TypeScript + Vite y portal externo React 19 + TypeScript + Vite.
- El backend expone una API REST protegida bajo /api, junto con rutas publicas para registro de empresas y rutas de portal basadas en token.
- El panel interno consume la API mediante un cliente ligero (frontend/app/src/services/api.ts) con cabecera Basic automatica desde variables de entorno.
- El portal externo permite registrar empresas, verificar el correo con token y usar un canal de mensajeria asociado a la solicitud.
- La persistencia se resuelve con Doctrine ORM y SQLite por defecto en desarrollo, aunque la configuracion admite otras bases soportadas por Symfony.

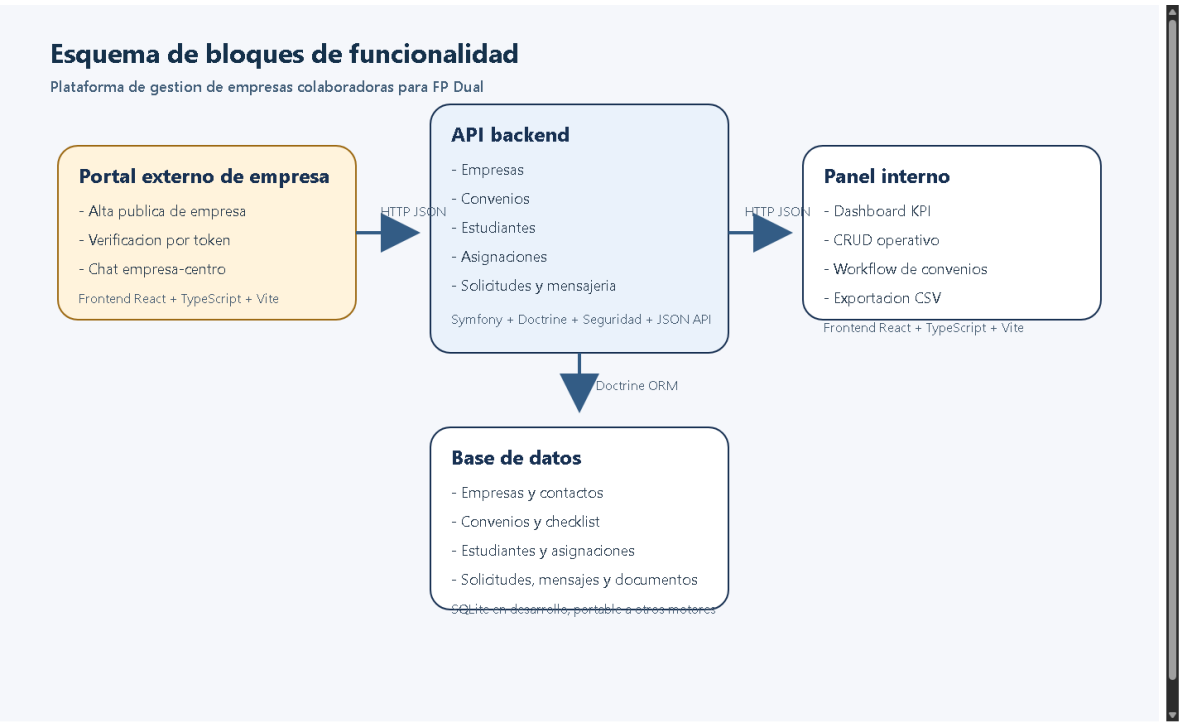

Figura 1. Esquema de bloques de funcionalidad del sistema.

9.2.1 Justificacion tecnica de versiones y separacion

- Existen dos versiones de React porque el panel interno y el portal externo son dos SPA independientes y se han construido en iteraciones distintas.
- El panel interno se mantiene en React 18.3.1 porque arranca desde una base anterior ya estabilizada para el modulo de gestion.
- El portal externo usa React 19.2.0 porque se creo despues con la plantilla actual de Vite.
- Esta diferencia no provoca conflictos de compatibilidad al no compartir bundle ni dependencias en runtime.

9.2.2 HTTPS como configuracion global

- El backend usa DEFAULT_URI para generar enlaces absolutos coherentes con el esquema final del despliegue.
- CORS_ALLOW_ORIGIN admite expresiones regulares y permite autorizar tanto http como https en entorno local.
- Ambos frontends pueden activar HTTPS en Vite mediante variables de entorno (VITE_DEV_HTTPS, VITE_DEV_HTTPS_KEY, VITE_DEV_HTTPS_CERT).
- De este modo la aplicacion puede demostrarse por HTTP en local simple o por HTTPS cuando se disponga de certificados de desarrollo.

9.3 Modelo de datos

- Entidades nucleares: EmpresaColaboradora, Convenio, Estudiante, TutorAcademico, TutorProfesional, AsignacionPractica, Seguimiento y EvaluacionFinal.
- Entidades de soporte a negocio: EmpresaSolicitud, EmpresaMensaje, EmpresaDocumento, EmpresaEtiqueta, EmpresaNota, ConvenioDocumento, ConvenioChecklistItem, ConvenioAlerta y ConvenioWorkflowEvento.
- Seguridad y acceso: entidad User con roles ROLE_USER, ROLE_API y ROLE_ADMIN.
- Relaciones principales: una empresa tiene convenios, contactos, documentos, notas y tutores; un convenio genera asignaciones y eventos de workflow; cada asignacion enlaza estudiante, empresa y tutores, y produce seguimientos y evaluacion final.

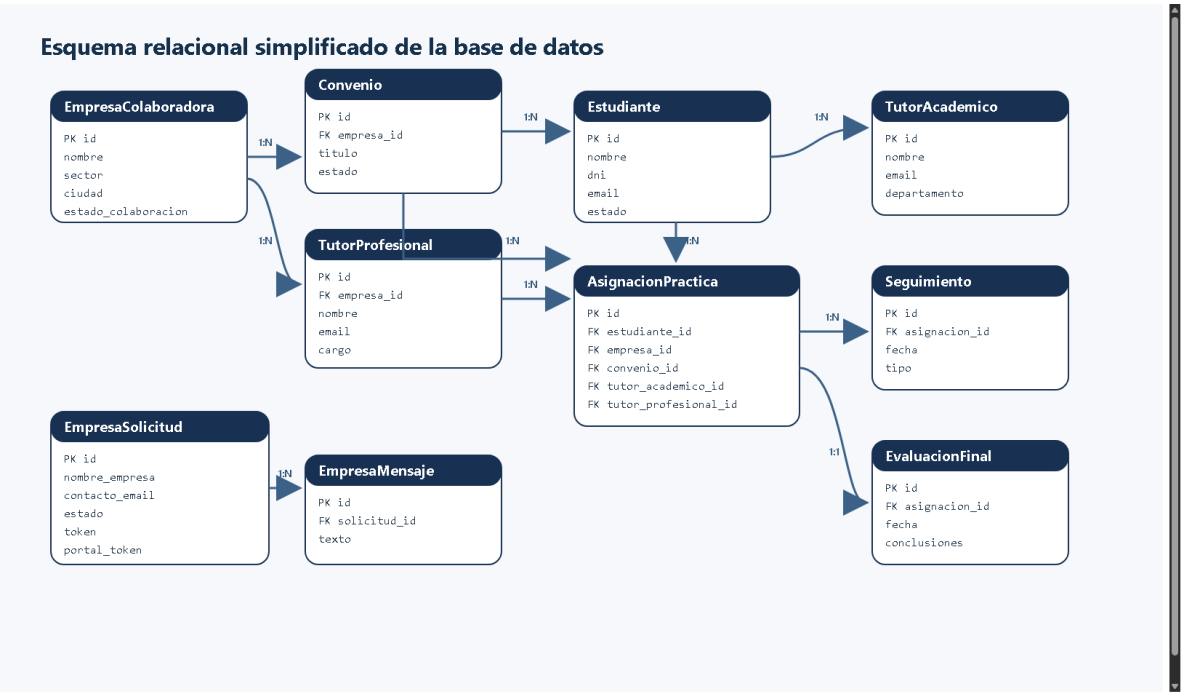

Figura 2. Esquema relacional de la base de datos.

9.4 Seguridad y cumplimiento

- La API usa autenticacion HTTP Basic y tambien deja preparado json_login para integraciones futuras.

- La jerarquia de seguridad define ROLE_ADMIN por encima de ROLE_API y ROLE_USER.
- Las rutas publicas se limitan al alta de solicitudes y a la confirmacion de correo o token; el resto de recursos de gestion requieren autenticacion.
- El panel interno importa VITE_API_BASE_URL, VITE_API_USERNAME y VITE_API_PASSWORD desde .env.local, evitando credenciales dispersas en el codigo.
- Las subidas de documentos reutilizan la misma autenticacion de entorno para mantener un comportamiento coherente en todas las llamadas a la API.
- En cumplimiento y privacidad, el sistema trabaja con datos minimos necesarios, separa portal publico y panel interno, y deja como mejora futura la auditoria avanzada y la politica definitiva de conservacion documental.

9.5 Diseno de interfaz

- El panel principal se organiza en modulos: dashboard, empresas, convenios, estudiantes, asignaciones, tutores, solicitudes y una guia de documentacion separada del centro privado de control y del monitor operativo.
- La interfaz combina tablas, tarjetas de detalle, chips de estado, modales de formulario, toasts y un centro de notificaciones para solicitudes pendientes.
- La vista de convenios incluye checklist, workflow, documentos y alertas activas o descartadas.
- La vista de empresas concentra KPI, convenios asociados, asignaciones, notas, etiquetas y documentos.
- El portal externo ofrece una experiencia diferenciada y mas simple: formulario de alta, bandeja de verificacion y chat empresa-centro.

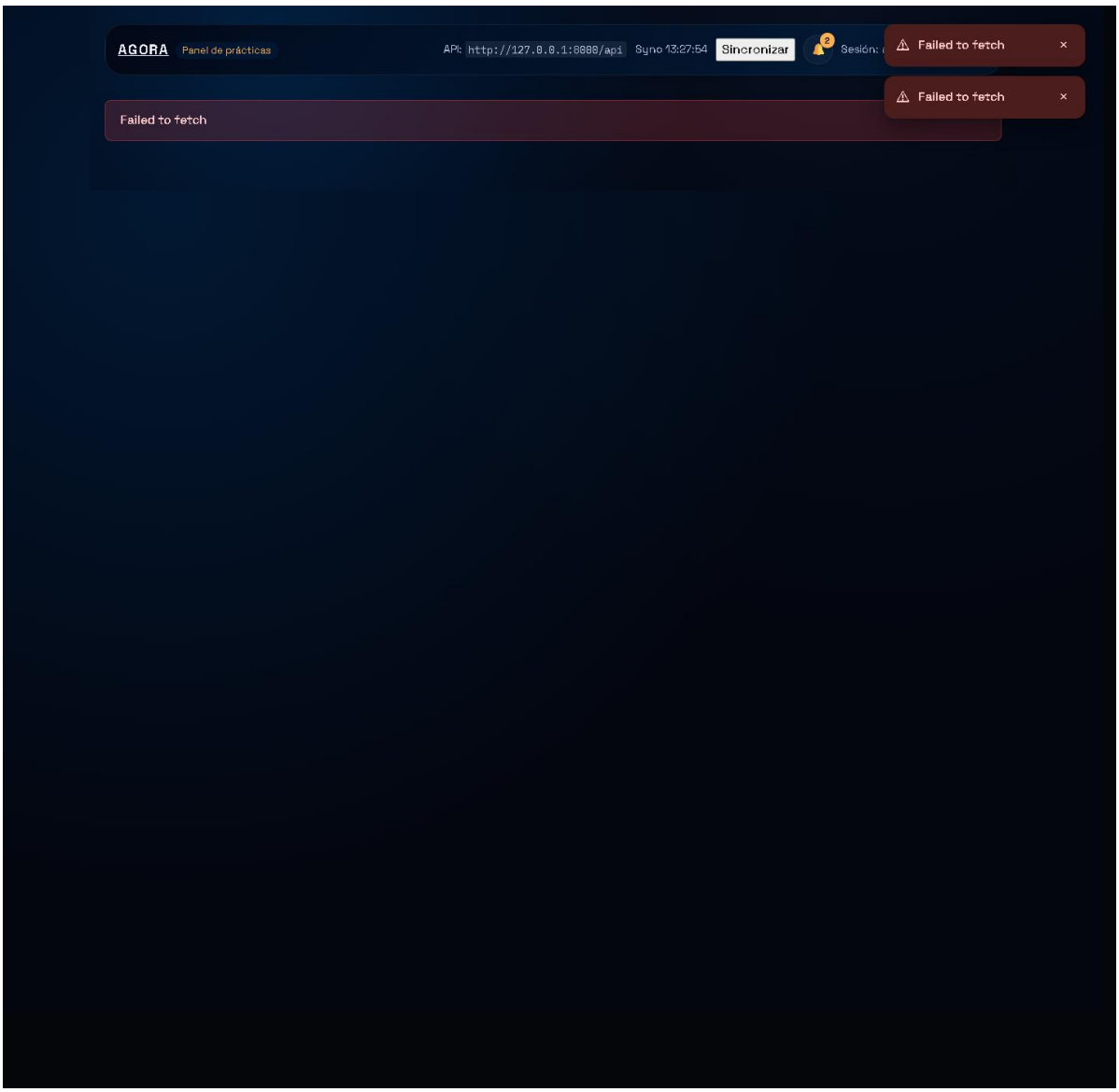

Figura 3. Panel interno, vista dashboard con KPI y exportacion.

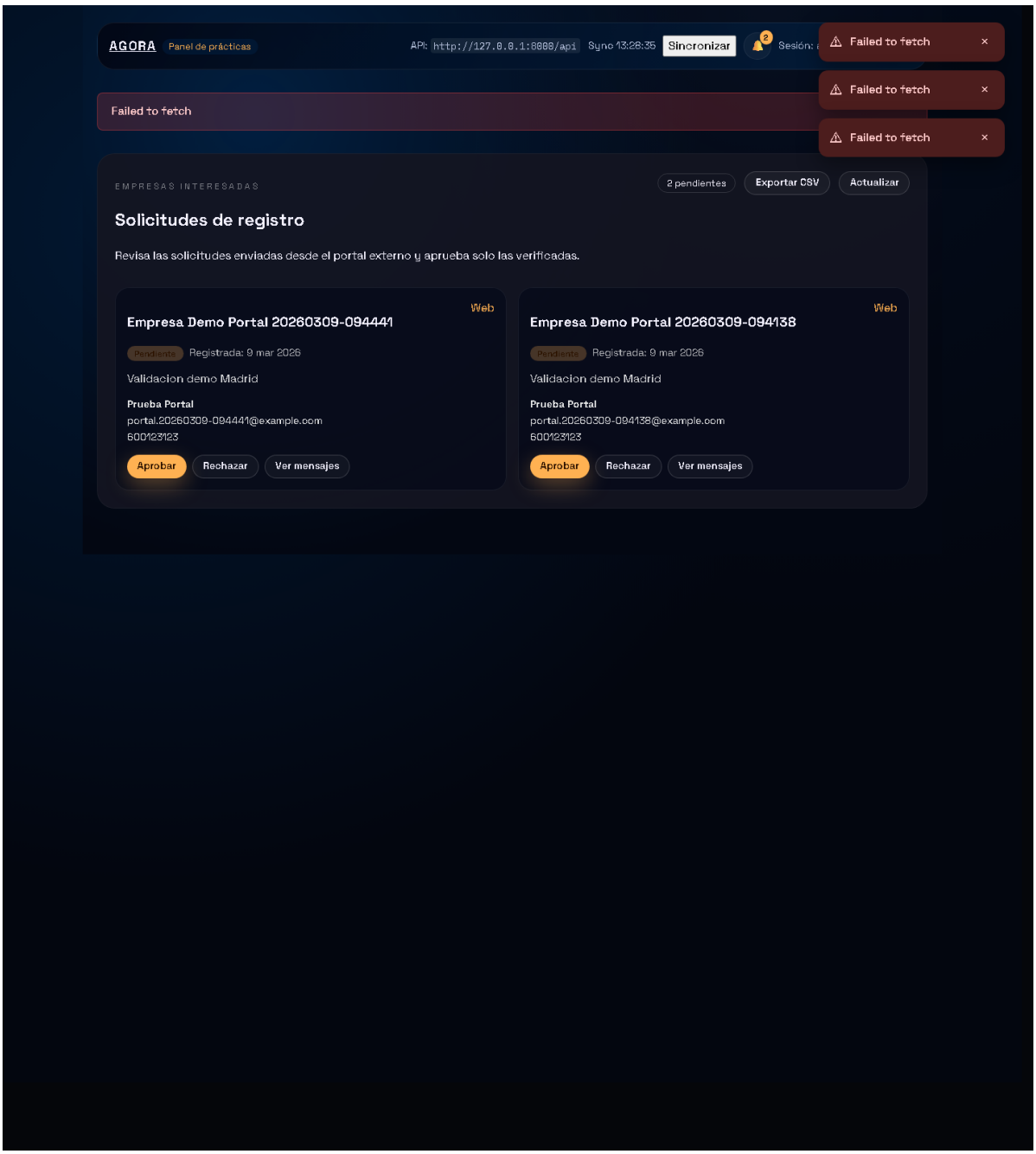

Figura 4. Panel interno, vista de solicitudes de empresa.

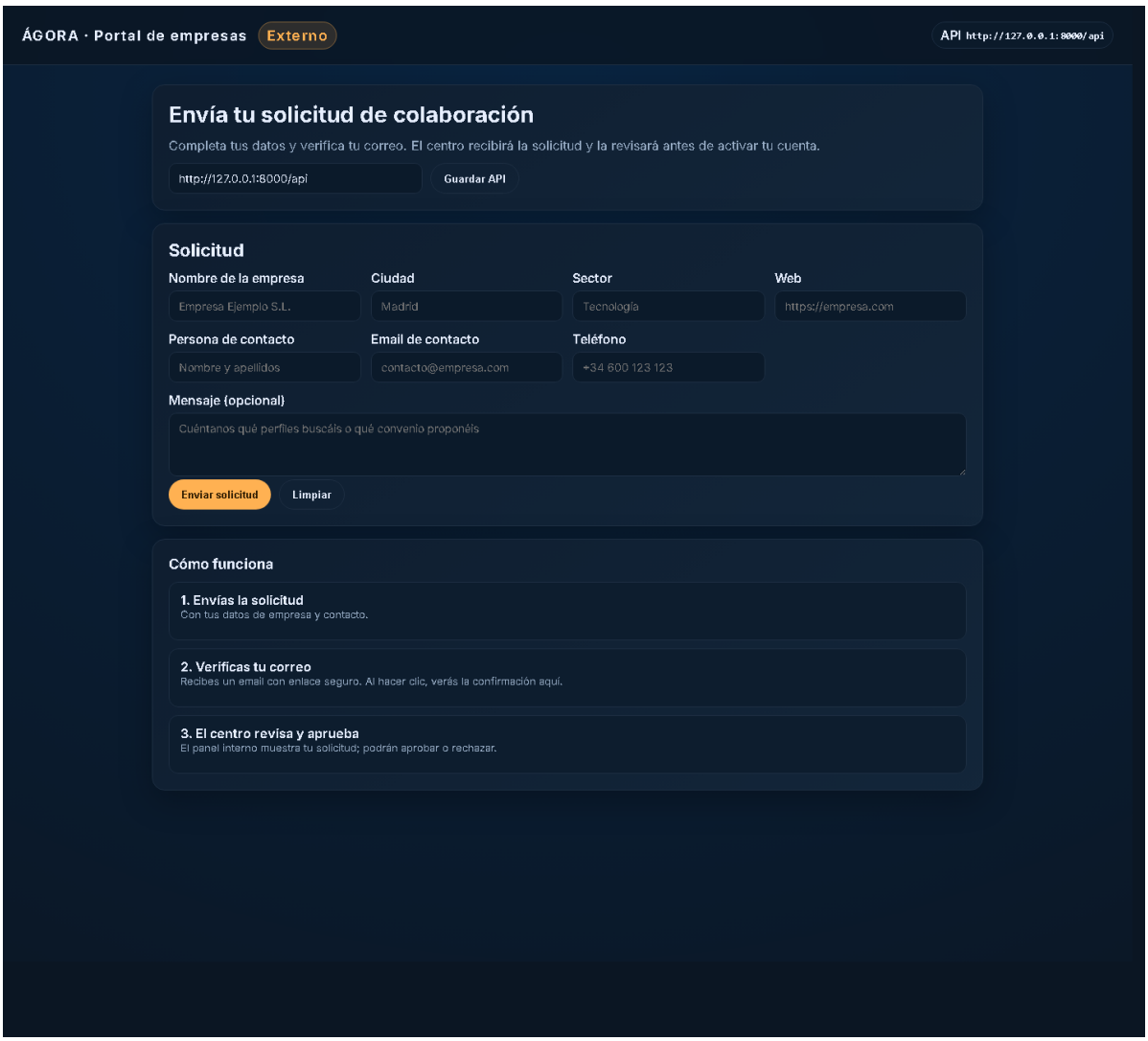

Figura 5. Portal externo para registro y seguimiento de solicitudes.

9.6 Implementacion

- Backend:

- Controladores REST para empresas, convenios, estudiantes, asignaciones y tutores. - Controladores especificos para solicitudes de empresa, mensajeria asociada y portal por token. - Fixtures demo con usuarios, empresas, convenios, estudiantes, tutores, asignaciones, checklist, alertas y mensajes.

- Frontend interno:

- App.tsx actua como shell del panel, dashboard y modulos de detalle. - Formularios modales reutilizables para empresas, convenios, estudiantes y asignaciones. - Cliente API con helpers apiGet, apiPost, apiPut, PATCH, soporte para subida de documentos y descarga de CSV desde endpoints dedicados. - Separacion entre guia funcional (/app/documentacion), centro privado de control (/app/control) y monitor operativo (/app/monitor).

- Frontend externo:

- Landing con formulario publico. - Confirmacion de email mediante token. - Chat simple sobre la solicitud mientras esta en revision.

- Estado actual de implementacion: la funcionalidad principal esta operativa y validada; el panel ya incorpora exportacion CSV de resumenes y listados, y el modulo de convenios persiste workflow, checklist y alertas mediante la API.

9.6.1 Ejemplo funcional implementado: exportacion CSV

El panel interno incorpora exportacion CSV como funcionalidad transversal. En la version actual conviven dos flujos complementarios:

1. El dashboard genera un resumen CSV desde frontend para exportar rapidamente los KPI principales.
2. Los listados operativos de empresas, convenios, estudiantes, asignaciones, tutores y solicitudes consumen endpoints CSV especificos del backend.
3. El frontend invoca esas rutas, descarga el blob y asigna un nombre de fichero coherente para la evidencia.
4. El resultado permite conservar una traza operativa externa a la aplicacion, util para seguimiento, revision academica o defensa del proyecto.

9.7 Pruebas y validacion

- El backend dispone de una suite automatizada con 61 pruebas y 344 aserciones.
- La bateria cubre autenticacion, listado y detalle de empresas, convenios, estudiantes y asignaciones, flujo completo de solicitudes, portal por token, documentos y repositorios.
- El 17/03/2026 se verifico ademas el arranque real de los servicios locales integrados:

- API Symfony en http://127.0.0.1:8000 - Panel interno en http://127.0.0.1:8000/app - Portal externo en http://127.0.0.1:8000/externo - Guia de documentacion en http://127.0.0.1:8000/app/documentacion - Centro privado de control en http://127.0.0.1:8000/app/control - Monitor operativo en http://127.0.0.1:8000/app/monitor

- Se valido tambien que la documentacion local sigue accesible aunque el acceso publico temporal se encuentre apagado.
- El frontend genera build de produccion sin errores con npm run build y npm run build:backend.
- Como limitacion de validacion, todavia no existe una suite E2E de navegador.

9.8 Despliegue y operacion

- Tras descargar el proyecto desde Git no queda funcional de forma inmediata: es necesario ejecutar composer install, npm install y preparar los .env.local.
- En desarrollo local se usa backend/.env.local con SQLite y los .env.local de ambos frontends para URL base, credenciales y HTTPS opcional.
- El backend puede iniciarse con Symfony CLI o servidor PHP local; para la defensa se utiliza una build integrada en modo URL unica bajo /app y /externo.
- La operacion cotidiana requiere unicamente PHP, Composer, Node.js, npm y la base SQLite local.
- El acceso externo temporal puede exponerse mediante tunel Cloudflare, gestionado desde el centro privado de control.

- Para una entrega productiva se recomienda separar frontends estaticos, API Symfony y almacenamiento documental, ademas de sustituir Basic auth por un mecanismo mas robusto.

9.9 Resultados y metricas (17/03/2026)

- Servicios locales levantados y accesibles de forma concurrente: backend, panel interno integrado, portal externo, guia de documentacion, centro privado y monitor.
- La llamada autenticada a /api/empresas devuelve respuesta valida y datos de demo.
- php bin/phpunit finaliza en verde con 61 pruebas y 344 aserciones.
- npm test, npm run build y npm run build:backend se completan correctamente en frontend/app.
- El panel interno permite exportar a CSV el dashboard, empresas, convenios, estudiantes, asignaciones, tutores y solicitudes.
- El flujo funcional mas relevante queda cubierto: solicitud externa, confirmacion por token, aprobacion interna, consulta de detalle, workflow de convenios persistido, gestion documental basica y supervision de acceso publico.

9.10 Limitaciones y riesgos

- La autenticacion actual resuelve el entorno academico y de demo, pero no cubre escenarios de SSO, MFA o auditoria avanzada.
- El modulo de asignaciones expone seguimientos y evaluacion final en consulta, pero no incorpora aun un CRUD especifico completo para editar estas piezas desde la API y el panel.
- El modulo de tutores se entrega como consulta operativa con exportacion, pero no como gestion CRUD completa de tutores academicos y profesionales.
- Las entidades principales no disponen todavia de un ciclo completo de archivado o baja logica con endpoints dedicados de cierre de vida.
- El portal externo cubre alta, verificacion y chat, pero no incluye aun un area postaprobacion para empresa ya validada.
- No existe integracion con sistemas corporativos externos ni firma electronica avanzada.
- El almacenamiento de documentos sigue siendo local y debe endurecerse antes de usar datos reales.
- Falta automatizar pruebas E2E del panel y del portal para reducir riesgo de regresion visual o de flujo.

9.11 Verificacion tecnica (17/03/2026)

- Backend:

- php bin/phpunit -> OK, 61 tests, 344 assertions. - symfony server:start --no-tls -d --port=8000 -> servicio operativo.

- Frontend interno:

- npm test -> validacion de utilidades y endpoints de control. - npm run build -> compilacion correcta. - npm run build:backend -> build integrada publicada en backend/public/app.

- Portal externo:

- GET /externo en la URL integrada -> 200 OK.

- Verificacion HTTP:

- GET /api/empresas con admin/admin123 -> 200 OK. - GET /app, /app/documentacion, /app/control y /app/monitor -> 200 OK. - GET /api/monitor y GET /api/public-access -> 200 OK.

10. Conclusiones y recomendaciones

- El proyecto ya cumple el objetivo principal de centralizar la gestion de empresas colaboradoras y practicas en una plataforma unica y operativa.
- La separacion entre panel interno, portal externo y API permite escalar funcionalidades sin mezclar responsabilidades.
- El mayor valor aportado al centro es la trazabilidad del ciclo completo: solicitud, aprobacion, convenio, asignacion, seguimiento y documentacion.
- Para una siguiente iteracion se recomienda, por este orden, completar el CRUD de seguimientos y evaluacion final, ampliar la gestion de tutores, incorporar archivado o baja logica de entidades, habilitar un portal postaprobacion para empresas y automatizar pruebas E2E.
- A nivel de explotacion real tambien seria recomendable reforzar almacenamiento documental, seguridad avanzada e integraciones corporativas.

11. Referencias

1. Proyecto TFG Agora. docs/domain-model.md.
2. Proyecto TFG Agora. docs/guia-demo.md.
3. Symfony. Symfony Documentation.
4. Doctrine Project. Doctrine ORM Documentation.
5. React Team. React Documentation.
6. Parlamento Europeo y Consejo de la Union Europea. Reglamento (UE) 2016/679.

12. Bibliografia

- Documentacion oficial de Vite y TypeScript para la construccion de SPA modernas.
- Material docente del modulo de proyecto y guias internas del centro.
- Referencias sobre gestion de practicas, modelado REST y seguridad web.

13. Anexos

Anexo A. Manual de usuario

Referencia principal: docs/anexo-a-manual-usuario.md.

Anexo B. Manual tecnico

Referencia principal: docs/anexo-b-manual-tecnico.md.

Anexo C. Evidencias de validacion

Referencia principal: docs/anexo-c-capturas-y-evidencias.md.

Anexo D. Artefactos de apoyo

Referencias principales:

- docs/anexo-d-codigo-relevante.md
- docs/domain-model.md
- docs/refactor-plan.md

Anexo E. Defensa del proyecto

Referencias principales:

- docs/anexo-e-defensa.md
- docs/guion-defensa.md
- docs/preguntas-defensa.md

Anexo A. Manual de Usuario

1. Objetivo

Este anexo describe el uso funcional de la plataforma "Gestion de Empresas Colaboradoras" desde el punto de vista del usuario interno del centro y del contacto externo de empresa.

2. Perfiles de uso

- ROLE_ADMIN: administracion general del panel y acceso completo a recursos API.
- ROLE_API: operativa de gestion sobre empresas, convenios, estudiantes, asignaciones y solicitudes.
- Empresa externa: uso del portal publico para registrar su interes, verificar el correo y comunicarse con el centro.

3. Acceso al sistema

3.1 Panel interno

- URL: http://localhost:5173
- Credenciales demo:

- admin / admin123 - coordinador / coordinador123

3.2 Portal externo

- URL: http://localhost:5174
- No requiere autenticacion previa para registrar una solicitud.

4. Navegacion principal del panel interno

El panel se estructura en los siguientes modulos:

- Dashboard
- Empresas
- Convenios
- Estudiantes
- Asignaciones
- Tutores
- Solicitudes
- Documentacion

5. Dashboard

Desde el dashboard se puede:

- consultar KPI principales del sistema;

- revisar tarjetas resumen por modulo;
- abrir accesos rapidos a las areas operativas;
- exportar un CSV de resumen con indicadores y analitica.

6. Gestion de empresas

En el modulo de empresas el usuario puede:

- ver el listado general;
- crear una nueva empresa;
- editar una empresa existente;
- consultar la ficha 360;
- revisar convenios asociados;
- revisar asignaciones vinculadas;
- consultar notas, etiquetas y documentos;
- exportar el listado visible a CSV.

7. Gestion de convenios

En el modulo de convenios el usuario puede:

- listar convenios por empresa o estado;
- crear y editar convenios;
- revisar workflow y checklist documental;
- adjuntar documentos de apoyo;
- revisar alertas activas;
- exportar el listado visible a CSV.

8. Gestion de estudiantes

En el modulo de estudiantes el usuario puede:

- listar estudiantes registrados;
- dar de alta y editar fichas;
- revisar estado academico y asignaciones;
- consultar seguimiento resumido;
- exportar el listado visible a CSV.

9. Gestion de asignaciones

En el modulo de asignaciones el usuario puede:

- consultar el pipeline completo;
- filtrar por estado y modalidad;
- crear nuevas asignaciones;
- editar asignaciones existentes;
- abrir la ficha de detalle;
- exportar el listado visible a CSV.

10. Gestion de tutores

En el modulo de tutores el usuario puede:

- consultar tutores academicos;
- consultar tutores profesionales;
- refrescar datos paginados;
- exportar cada tabla a CSV.

11. Solicitudes de empresa

En el modulo de solicitudes el usuario puede:

- revisar nuevas solicitudes enviadas desde el portal;
- aprobar solicitudes verificadas;
- rechazar solicitudes indicando motivo;
- consultar e intercambiar mensajes con la empresa;
- exportar el listado visible a CSV.

12. Portal externo

El flujo del portal externo es:

1. La empresa completa el formulario de solicitud.
2. Recibe un enlace de verificacion por correo.
3. Confirma el token desde la vista de verificacion.
4. Usa el canal de mensajes mientras su solicitud esta en revision.

13. Errores y validaciones comunes

- Si el backend no responde, el panel mostrara mensajes de error y no cargara las colecciones.
- Si faltan credenciales o son incorrectas, la API devolvera error de autenticacion.
- Si un formulario contiene datos invalidos, el modal mostrara el mensaje correspondiente.

14. Recomendaciones de uso

- Refrescar solicitudes antes de aprobar o rechazar.

- Revisar el estado documental de convenios antes de avanzar workflow.

- Exportar CSV como apoyo para revision, seguimiento y defensa del TFG.

Anexo B. Manual Tecnico

1. Arquitectura general

La solucion se compone de tres aplicaciones:

- backend/: API Symfony 7.3 con Doctrine ORM y SQLite por defecto.
- frontend/app/: panel interno React 18 + TypeScript + Vite.
- frontend/company-portal/: portal externo React 19 + TypeScript + Vite.

2. Requisitos locales

- PHP 8.2
- Composer 2.x
- Node.js 18+
- npm
- SQLite
- Symfony CLI opcional

3. Arranque del entorno

El repositorio no queda operativo solo con clonar desde Git. Hace falta instalar dependencias y crear los .env.local.

3.1 Backend

```
cd backend
copy .env.local.example .env.local
composer install
php bin/console doctrine:migrations:migrate --no-interaction
php bin/console doctrine:fixtures:load --no-interaction
symfony server:start --no-tls -d --port=8000
```

3.2 Frontend interno

```
cd frontend/app
copy .env.example .env.local
npm install
npm run dev -- --host --port 5173 --strictPort
```

3.3 Portal externo

```
cd frontend/company-portal
copy .env.example .env.local
npm install
```

```
npm run dev -- --host --port 5174 --strictPort
```

4. Variables de entorno relevantes

4.1 Backend

- DATABASE_URL
- MAILER_DSN
- APP_MAIL_FROM
- DEFAULT_URI
- CORS_ALLOW_ORIGIN

4.2 Frontend interno

- VITE_API_BASE_URL
- VITE_API_USERNAME
- VITE_API_PASSWORD
- VITE_DEV_HTTPS
- VITE_DEV_HTTPS_KEY
- VITE_DEV_HTTPS_CERT

4.3 Portal externo

- VITE_API_BASE_URL
- VITE_DEV_HTTPS
- VITE_DEV_HTTPS_KEY
- VITE_DEV_HTTPS_CERT

5. Seguridad

- El backend protege /api mediante seguridad Symfony.
- Se usa HTTP Basic y json_login queda preparado para evolucion posterior.
- Las credenciales del panel se importan desde frontend/app/.env.local.
- Las subidas de documentos reutilizan esas mismas credenciales de entorno.
- El soporte HTTPS queda preparado mediante DEFAULT_URI, CORS_ALLOW_ORIGIN y configuracion TLS opcional en ambos Vite.
- Jerarquia de roles:

- ROLE_ADMIN - ROLE_API - ROLE_USER

6. Versiones de React

- frontend/app: React 18.3.1.

- frontend/company-portal: React 19.2.0.

La coexistencia es valida porque ambas aplicaciones son independientes, se compilan por separado y no comparten runtime en navegador.

7. Persistencia

La base de datos por defecto es SQLite. El esquema actual se crea a partir de la migracion base:

- backend/migrations/Version20251113203450.php

La carga de datos demo se realiza mediante:

- backend/src/DataFixtures/DemoDominioFixtures.php

8. Estructura funcional del backend

Controladores principales:

- backend/src/Controller/Api/EmpresaColaboradoraController.php
- backend/src/Controller/Api/ConvenioController.php
- backend/src/Controller/Api/EstudianteController.php
- backend/src/Controller/Api/AsignacionController.php
- backend/src/Controller/Api/TutorAcademicoController.php
- backend/src/Controller/Api/TutorProfesionalController.php
- backend/src/Controller/Api/EmpresaSolicitudController.php
- backend/src/Controller/Api/EmpresaMensajeController.php
- backend/src/Controller/RegistroEmpresaController.php
- backend/src/Controller/Portal/SolicitudPortalController.php

9. Estructura funcional del frontend interno

Archivos clave:

- frontend/app/src/App.tsx
- frontend/app/src/components/DataTable.tsx
- frontend/app/src/components/EmpresaForm.tsx
- frontend/app/src/components/ConvenioForm.tsx
- frontend/app/src/components/EstudianteForm.tsx
- frontend/app/src/components/AsignacionForm.tsx
- frontend/app/src/services/api.ts
- frontend/app/src/utils/csv.ts

10. Exportacion CSV

La exportacion CSV se realiza en cliente mediante:

- frontend/app/src/utils/csv.ts

Actualmente se exportan:

- dashboard;
- empresas;
- convenios;
- estudiantes;
- asignaciones;
- tutores;
- solicitudes.

11. Validacion tecnica

Comandos principales:

```
cd backend
php bin/phpunit
cd frontend/app
npm run build
cd frontend/company-portal
npm run build
```

12. Riesgos tecnicos abiertos

- no existe suite E2E de navegador;
- la autenticacion del panel sigue basada en credenciales de entorno;
- la salida PDF final de la memoria requiere maquetacion posterior;
- el almacenamiento documental sigue siendo local.

Anexo C. Capturas y Evidencias

1. Capturas obligatorias recomendadas

Tomar una captura final de cada una de estas vistas:

1. Dashboard principal con KPI y analitica.
2. Modulo de empresas con listado y ficha 360 abierta.
3. Modulo de convenios con workflow, checklist y alertas.
4. Modulo de estudiantes con ficha academica y asignaciones.
5. Modulo de asignaciones con pipeline y detalle.
6. Modulo de tutores con ambas tablas visibles.
7. Modulo de solicitudes con mensajes y acciones de aprobacion/rechazo.
8. Portal externo con formulario de alta.
9. Portal externo con pantalla de verificacion de token.
10. Portal externo con canal de mensajes.
11. Guia de documentacion con recorrido funcional.
12. Centro privado de control o monitor operativo, solo como apoyo tecnico.

2. Evidencias tecnicas a incluir en anexos

- salida de php bin/phpunit;
- salida de npm run build en frontend/app;
- salida de npm run build en frontend/company-portal;
- comprobacion de GET /api/empresas con respuesta 200 OK;
- comprobacion visual de exportacion CSV.

3. Nombre recomendado para las capturas

- 01-dashboard.png
- 02-empresas.png
- 03-convenios.png
- 04-estudiantes.png
- 05-asignaciones.png
- 06-tutores.png
- 07-solicitudes.png
- 08-portal-alta.png

- 09-portal-verificacion.png
- 10-portal-chat.png
- 11-documentacion-guia.png
- 12-control-monitor.png

4. Ubicacion sugerida

Guardar las capturas finales en:

- docs/capturas/

5. Indice actual y objetivo de capturas

- 01-bloques-funcionalidad.png: vista general de bloques funcionales.
- 02-esquema-relacional.png: esquema relacional de entidades y tablas.
- 03-panel-interno-dashboard.png: dashboard del panel interno.
- 04-panel-interno-solicitudes.png: modulo de solicitudes del panel interno.
- 05-portal-externo.png: portada operativa del portal externo.
- 06-documentacion-guia.png: guia funcional de la plataforma. Pendiente de refresco visual final.
- 07-centro-control.png: panel privado para acceso publico y soporte de demo. Pendiente de refresco visual final.
- 08-monitor-operativo.png: supervision operativa, tests y logs. Pendiente de refresco visual final.
- 03-panel-interno-dashboard.html: copia HTML de apoyo para la captura 03.
- 04-panel-interno-solicitudes.html: copia HTML de apoyo para la captura 04.

6. Comprobacion antes de incrustar en la memoria

- verificar que la fecha y hora del sistema no distraigan;
- ocultar credenciales o datos irrelevantes;
- mantener una resolucion suficiente para impresion;
- usar siempre el mismo zoom y ancho de ventana;
- capturar tambien una exportacion CSV descargada como prueba funcional.
- comprobar que la guia publica y el centro privado son paginas distintas antes de capturar.

Anexo D. Codigo Relevante

1. Backend

Seguridad y autenticacion

- backend/config/packages/security.yaml
- backend/src/Controller/Api/AuthController.php
- backend/src/Entity/User.php

Dominio y persistencia

- backend/src/Entity/EmpresaColaboradora.php
- backend/src/Entity/Convenio.php
- backend/src/Entity/Estudiante.php
- backend/src/Entity/AsignacionPractica.php
- backend/src/Entity/EmpresaSolicitud.php
- backend/migrations/Version20251113203450.php
- backend/src/DataFixtures/DemoDominioFixtures.php

Casos de uso principales

- backend/src/Controller/Api/EmpresaColaboradoraController.php
- backend/src/Controller/Api/ConvenioController.php
- backend/src/Controller/Api/EstudianteController.php
- backend/src/Controller/Api/AsignacionController.php
- backend/src/Controller/Api/EmpresaSolicitudController.php
- backend/src/Controller/Portal/SolicitudPortalController.php

2. Frontend interno

Shell principal y modulos

- frontend/app/src/App.tsx

Formularios reutilizables

- frontend/app/src/components/EmpresaForm.tsx
- frontend/app/src/components/ConvenioForm.tsx
- frontend/app/src/components/EstudianteForm.tsx
- frontend/app/src/components/AsignacionForm.tsx

Infraestructura de UI

- frontend/app/src/components/DataTable.tsx
- frontend/app/src/components/Modal.tsx
- frontend/app/src/components/ToastStack.tsx

Cliente API y exportacion

- frontend/app/src/services/api.ts
- frontend/app/src/utils/csv.ts

3. Frontend externo

- frontend/company-portal/src/App.tsx

4. Suite de pruebas destacada

- backend/tests/Security/AuthenticationTest.php
- backend/tests/Controller/Portal/SolicitudPortalControllerTest.php
- backend/tests/Controller/Api/EmpresaSolicitudFlowTest.php
- backend/tests/Controller/Api/ConvenioControllerTest.php
- backend/tests/Controller/Api/AsignacionControllerTest.php
- backend/tests/Controller/Api/EmpresaControllerTest.php

5. Criterio de seleccion

Este anexo recoge las piezas mas representativas para explicar:

- arquitectura;
- seguridad;
- modelo de dominio;
- flujos de negocio;
- validacion tecnica;
- exportacion funcional.

Anexo E. Defensa del Proyecto

1. Objetivo del anexo

Este anexo recoge material de apoyo para la exposicion oral del Trabajo Final de Grado. El enfoque no es solo explicar el proyecto, sino justificar el trabajo realizado, las decisiones tomadas y el aprendizaje obtenido.

2. Mensaje principal de la defensa

La idea central de la presentacion es que el proyecto resuelve un problema real del centro mediante una solucion operativa y, al mismo tiempo, demuestra capacidad personal para analizar necesidades, acotar alcance, implementar una arquitectura completa y documentarla.

3. Guion breve de exposicion

3.1 Apertura

El proyecto surge para resolver una gestion dispersa basada en hojas de calculo, correos y documentos aislados. La propuesta consiste en una plataforma unica que centraliza empresas colaboradoras, convenios y practicas formativas.

3.2 Problema

La gestion previa generaba duplicidades, poca trazabilidad, dependencia del conocimiento manual y escasa visibilidad del estado real de convenios, solicitudes y asignaciones.

3.3 Solucion

La solucion se ha dividido en tres piezas:

1. API REST en Symfony.
2. Panel interno en React y TypeScript.
3. Portal externo para empresas.

3.4 Aportacion personal

- Identificacion del problema y redefinicion del alcance.
- Modelado de datos y arquitectura separada por responsabilidades.
- Implementacion completa de backend y dos frontends.
- Validacion tecnica y redaccion de memoria, anexos y material de defensa.

3.5 Cierre

La conclusion defendible es que el TFG no solo entrega una aplicacion funcional, sino tambien una muestra clara del proceso seguido, de las decisiones tecnicas adoptadas y del aprendizaje adquirido.

4. Preguntas previsibles y linea de respuesta

4.1 Por que elegiste este tema?

Porque resolvia una necesidad concreta y permitia desarrollar una solucion util, no solo un ejercicio tecnico.

4.2 Que has hecho tu directamente?

Analisis, arquitectura, implementacion, validacion y documentacion principal del proyecto.

4.3 Que ha sido lo mas complicado?

Mantener el equilibrio entre alcance realista, coherencia tecnica y tiempo disponible.

4.4 Que mejorarias con una iteracion adicional?

Seguridad productiva, pruebas E2E, gestion completa de algunos modulos y endurecimiento del despliegue.

5. Orden recomendado para la demo

1. Dashboard y KPI.
2. Empresas y ficha 360.
3. Convenios con workflow, checklist y alertas.
4. Asignaciones y relaciones con estudiantes y tutores.
5. Solicitudes de empresa.
6. Portal externo.

6. Mensaje de cierre recomendado

El valor del TFG esta en haber convertido una necesidad real en una solucion completa, trazable y defendible, mostrando capacidad para analizar, decidir, implementar y justificar tecnicamente el trabajo realizado.